

COMPUTER SYSTEMS

A Programmer's Perspective

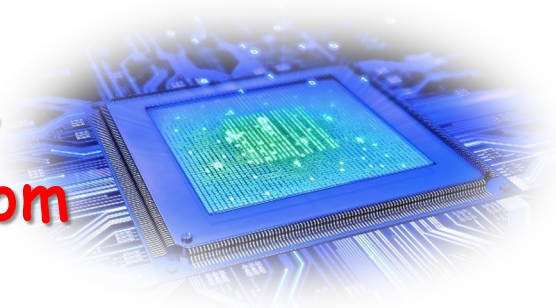
深入理解计算机系统



主讲人：张延松

中国人民大学 信息学院

zhangys_ruc@hotmail.com



前言 / INTRODUCTION



第7章 链接

教学目标：

- ◆ 理解链接器如何帮助构造大型程序
- ◆ 理解链接器如何帮助避免一些危险的编程错误
- ◆ 理解链接如何帮助理解语言的作用域规则实现
- ◆ 理解链接如何帮助理解其他重要的系统概念
- ◆ 理解链接如何利用共享库

本章学习要求

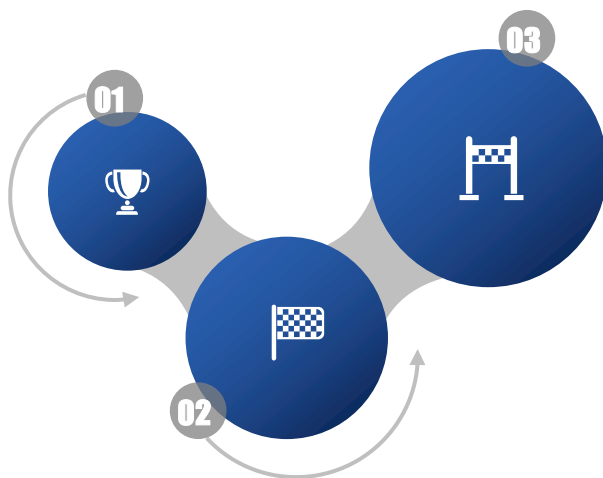
了解链接工作原理

掌握链接工作原理。

- 理解链接器如何帮助构造大型程序
- 理解符号解析如何帮助避免编程错误
- 理解语言作用域规则的实现方法
- 学习使用共享库

实践

通过反汇编查看符号解析



扩展

LLVM JIT实时编译技术在数据库中的应用

目标： 学习大型软件的分离编译方法

7.1 编译器驱动程序

- 示例：
 - Main函数初始化一个整数数组
 - Sum函数对元素求和

```
int sum(int *a, int n);

int array[2] = {1, 2};

int main()
{
    int val = sum(array, 2);
    return val;
}
```

main.c

```
int sum(int *a, int n)
{
    int i, s = 0;

    for (i = 0; i < n; i++) {
        s += a[i];
    }
    return s;
}
```

sum.c

静态链接示例

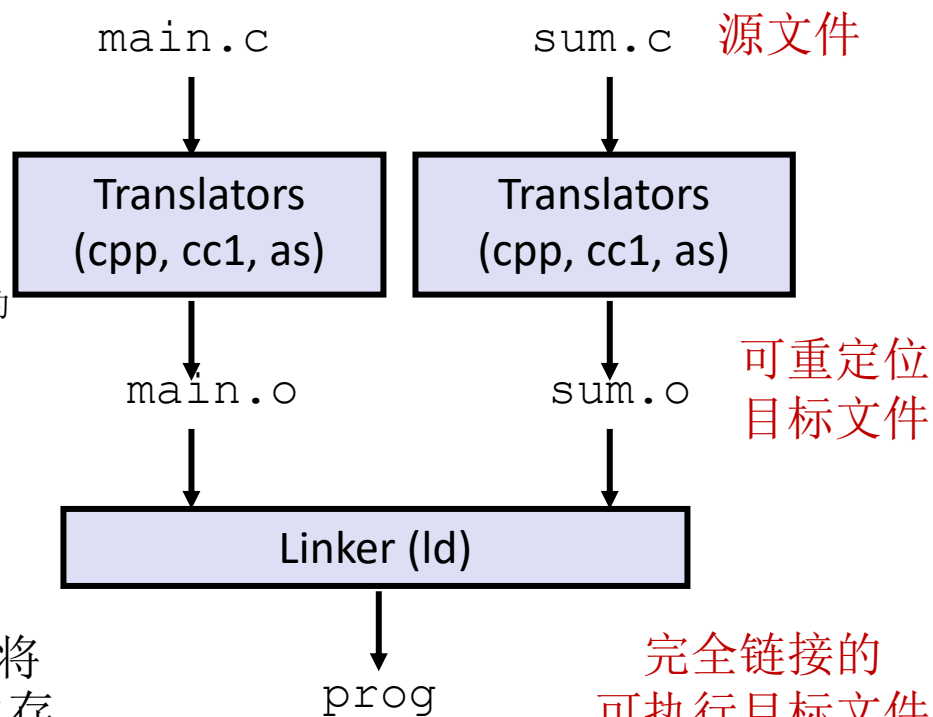
- 编译系统提供编译器驱动程序，代表用户时需要调用语言预处理器、编译器、汇编器和链接器：

```
linux> gcc -Og -o prog main.c sum.c
```

- 在shell中调用gcc驱动程序：

- 将源程序翻译成ASCII码中间文件
`cpp main.c /tmp/main.i`
- 运行C编译器，将main.i翻译成ASCII汇编语言文件
`cc1 /tmp/main.i -Og -o /tmp/main.s`
- 运行汇编器，将main.s翻译成可重定位目标文件
`as -o /tmp/main.o /tmp/main.s`
- 运行链接器程序，将main.o和sum.o以及一些必要的系统目标文件组合起来，创建可执行目标文件
`ld -o prog /tmp/main.o /tmp/sum.o`

```
linux> ./prog
```



Shell调用操作系统中加载器（loader），将可执行文件prog中的代码和数据复制到内存，然后将控制转移到这个程序的开头

实验验证

- gcc -o3 -o prog main.c sum.c

```
#include <stdlib.h>
#include <stdio.h>

int sum(int *a, int n);
int array[2]={1,2};
int main()
{
    int val=sum(array, 2);
    printf("sum of array:%d\n",val);
    return val;
}
```

```
int sum(int *a, int
n)
{
    int i,s=0;
    for(i=0;i<n;i+
+){
        s+=a[i];
    }
    return s;
}
```

```
[nsrc@localhost ~]$ gcc -o3 -o prog main.c sum.c
[nsrc@localhost ~]$ prog
sum of array:3
```

.c→.i

- C预处理器cpp将.c源程序翻译成.i ASCII中间文件
 - cpp main.c main.i
 - cpp sum.c sum.i
- 查看main.i中间文件

```
1 "main.c"
# 1 "<built-in>"
# 1 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 1 "<command-line>" 2
# 1 "main.c"
# 1 "/usr/include/stdlib.h" 1 3 4
# 24 "/usr/include/stdlib.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 375 "/usr/include/features.h" 3 4
# 1 "/usr/include/sys/cdefs.h" 1 3 4
# 392 "/usr/include/sys/cdefs.h" 3 4
# 1 "/usr/include/bits/wordsize.h" 1 3 4
# 393 "/usr/include/sys/cdefs.h" 2 3 4
# 376 "/usr/include/features.h" 2 3 4
# 399 "/usr/include/features.h" 3 4
# 1 "/usr/include/gnu/stubs.h" 1 3 4
# 10 "/usr/include/gnu/stubs.h" 3 4
# 1 "/usr/include/gnu/stubs-64.h" 1 3 4
# 11 "/usr/include/gnu/stubs.h" 2 3 4
# 400 "/usr/include/features.h" 2 3 4
# 25 "/usr/include/stdlib.h" 2 3 4

# 1 "/usr/lib/gcc/x86_64-redhat-linux/4.8.5/include/stddef.h" 1 3 4
```

.i→.s

- C编译器cc1将.i翻译成.s ASCII汇编语言文件
- 本例中cc1位置：
 - /usr/libexec/gcc/x86_64-redhat-linux/4.8.2/cc1
- /usr/libexec/gcc/x86_64-redhat-linux/4.8.2/cc1 main.i -O3 -o main.s
- /usr/libexec/gcc/x86_64-redhat-linux/4.8.2/cc1 sum.i -O3 -o sum.s

```
[nsrc@localhost ~]$ /usr/libexec/gcc/x86_64-redhat-linux/4.8.2/cc1 main.i -O3 -o main.s
__bswap_32 __bswap_64 main
Analyzing compilation unit
Performing interprocedural optimizations
<*free_lang_data> <visibility> <early_local_cleanups> <*free_inline_summary> <whole-program> <profile_estimate> <cp
> <inline> <pure-const> <static-var>Assembling functions:
main
Execution times (seconds)
phase setup      : 0.00 ( 0%) usr 0.00 ( 0%) sys 0.01 (25%) wall 1094 kB (62%) gcc
phase parsing    : 0.01 (50%) usr 0.01 (100%) sys 0.02 (50%) wall 583 kB (33%) gcc
phase opt and generate : 0.01 (50%) usr 0.00 ( 0%) sys 0.01 (25%) wall 57 kB ( 3%) gcc
preprocessing    : 0.01 (50%) usr 0.00 ( 0%) sys 0.01 (25%) wall 50 kB ( 3%) gcc
lexical analysis : 0.00 ( 0%) usr 0.01 (100%) sys 0.01 (25%) wall 0 kB ( 0%) gcc
integrated RA    : 0.01 (50%) usr 0.00 ( 0%) sys 0.00 ( 0%) wall 12 kB ( 1%) gcc
unaccounted todo : 0.00 ( 0%) usr 0.00 ( 0%) sys 0.01 (25%) wall 0 kB ( 0%) gcc
TOTAL           : 0.02      0.01      0.04      1752 kB
[nsrc@localhost ~]$ /usr/libexec/gcc/x86_64-redhat-linux/4.8.2/cc1 sum.i -O3 -o sum.s
sum
Analyzing compilation unit
Performing interprocedural optimizations
<*free_lang_data> <visibility> <early_local_cleanups> <*free_inline_summary> <whole-program> <profile_estimate> <cp
> <inline> <pure-const> <static-var>Assembling functions:
sum
Execution times (seconds)
phase setup      : 0.00 ( 0%) usr 0.00 ( 0%) sys 0.01 (25%) wall 1094 kB (69%) gcc
phase opt and generate : 0.03 (100%) usr 0.00 ( 0%) sys 0.03 (75%) wall 369 kB (23%) gcc
ipa cp           : 0.00 ( 0%) usr 0.00 ( 0%) sys 0.01 (25%) wall 0 kB ( 0%) gcc
tree copy propagation : 0.01 (33%) usr 0.00 ( 0%) sys 0.00 ( 0%) wall 1 kB ( 0%) gcc
CPRP             : 0.00 ( 0%) usr 0.00 ( 0%) sys 0.01 (25%) wall 6 kB ( 0%) gcc
if-conversion     : 0.01 (33%) usr 0.00 ( 0%) sys 0.00 ( 0%) wall 0 kB ( 0%) gcc
machine dep reorg : 0.01 (33%) usr 0.00 ( 0%) sys 0.00 ( 0%) wall 0 kB ( 0%) gcc
reorder blocks    : 0.00 ( 0%) usr 0.00 ( 0%) sys 0.01 (25%) wall 4 kB ( 0%) gcc
TOTAL           : 0.03      0.00      0.04      1585 kB
```

```
[nsrc@localhost ~]$ cat main.s
.file "main.i"
.section .rodata.str1.1,"aMS",@progbits,1
.LC0:
.string "sum of array:%d\n"
.section .text.startup,"ax",@progbits
.p2align 4,,15
.globl main
.type main, @function

main:
.LFB2:
.cfi_startproc
pushq %rbx
.cfi_def_cfa_offset 16
.cfi_offset 3, -16
movl $2, %esi
movl %array, %edi
call sum
movl $.LC0, %edi
movl %eax, %ebx
movl %eax, %esi
xorl %eax, %eax
call printf
movl %ebx, %eax
popq %rbx
.cfi_def_cfa_offset 8
ret
.LFE2:
.size main, .-main
.globl array
.data
.align 4
.type array, @object
.size array, 8
```

.s→.o

- 汇编器as将.s翻译成.o 可重定位目标件

- as -o main.o main.s

- as -o sum.o sum.s

```
[nsrc@localhost ~]$ as -o main.o main.s
[nsrc@localhost ~]$ as -o sum.o sum.s
```

- 链接器ld将main.o和sum.o以及一些必要的系统目标文件组合起来，创建可执行目标文件prog（链接错误，未生成）

- ld -o prog main.o sum.o

```
[nsrc@localhost ~]$ ld -o prog main.o sum.o
ld: warning: cannot find entry symbol _start; defaulting to 00000000004000f0
main.o: In function `main':
main.i:(.text.startup+0x1c): undefined reference to `printf'
```

- Gcc直接编译执行

- gcc -O3 -o prog main.c sum.c

```
[nsrc@localhost ~]$ gcc -O3 -o prog main.c sum.c
[nsrc@localhost ~]$ prog
sum of array:3
```

查看gcc编译

- gcc -O3 -o prog main.c sum.c -v

```
[nsrc@localhost ~]$ gcc -O3 -o prog main.c sum.c -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/libexec/gcc/x86_64-redhat-linux/4.8.5/lto-wrapper
Target: x86_64-redhat-linux
Configured with: ../configure --prefix=/usr --mandir=/usr/share/man --infodir=/usr/share/info --with-bugurl=http://bugzilla.redhat.com/bugzilla --enable-bootstrap --enable-shared --enable-threads=posix --enable-checking=release --with-system-zlib --enable-__cxa_atexit --disable-libunwind-exceptions --enable-gnu-unique-object --enable-linker-build-id --with-linker-hash-style=gnu --enable-languages=c,c++,objc,obj-c++,java,fortran,ada,go,lto --enable-plugin --enable-initfini-array --disable-libgomp --with-isl=/builddir/build/BUILD/gcc-4.8.5-20150702/obj-x86_64-redhat-linux/isl-install --with-cloog=/builddir/build/BUILD/gcc-4.8.5-20150702/obj-x86_64-redhat-linux/cloog-install --enable-gnu-indirect-function --with-tune=generic --with-arch_32=x86_64 --build=x86_64-redhat-linux
Thread model: posix
gcc version 4.8.5 20150623 (Red Hat 4.8.5-11) (GCC)
COLLECT_GCC_OPTIONS='-O3' '-o' 'prog' '-v' '-mtune=generic' '-march=x86-64'
/usr/libexec/gcc/x86_64-redhat-linux/4.8.5/ccl -quiet -v main.c -quiet -dumpbase main.c -mtune=generic -march=x86-64 -auxbase main -O3 -version -o /tmp/ccPNeDdd.s
GNU C (GCC) version 4.8.5 20150623 (Red Hat 4.8.5-11) (x86_64-redhat-linux)
    compiled by GNU C version 4.8.5 20150623 (Red Hat 4.8.5-11), GMP version 6.0.0, MPFR version 3.1.1, MPC version 1.0.1
GGC heuristics: --param ggc-min-expand=100 --param ggc-min-heapsize=131072
ignoring nonexistent directory "/usr/lib/gcc/x86_64-redhat-linux/4.8.5/include-fixed"
ignoring nonexistent directory "/usr/lib/gcc/x86_64-redhat-linux/4.8.5/../../../../x86_64-redhat-linux/include"
#include "..." search starts here:
#include <...> search starts here:
/usr/local/cuda-8.0/targets/x86_64-linux/include
/usr/lib/gcc/x86_64-redhat-linux/4.8.5/include
/usr/local/include
/usr/include
End of search list.
GNU C (GCC) version 4.8.5 20150623 (Red Hat 4.8.5-11) (x86_64-redhat-linux)
    compiled by GNU C version 4.8.5 20150623 (Red Hat 4.8.5-11), GMP version 6.0.0, MPFR version 3.1.1, MPC version 1.0.1
GGC heuristics: --param ggc-min-expand=100 --param ggc-min-heapsize=131072
Compiler executable checksum: 356f86e67978d665416e07d560c8ba0d
COLLECT_GCC_OPTIONS='-O3' '-o' 'prog' '-v' '-mtune=generic' '-march=x86-64'
as -v --64 -o /tmp/ccOfmFwh.o /tmp/ccPNeDdd.s
```

7.2 静态链接

- 静态链接器（static linker）：以一组可重定位目标文件和命令行参数作为输入，生成一个完全链接的、可以加载和运行的可执行目标为输出。
 - 输入的可重定位目标文件由各种不同的代码和数据节（section）组成，每一节都是一个连续的字节序列
 - 指令、初始化了的全局变量、未初始化的变量在不同的节中
- 为构造可执行文件，链接器必须完成的两个主要任务：
 - 符号解析：将每个符号引用和符号定义关联起来
 - 目标文件定义和引用符号，每个符号对应一个函数、一个全局变量或一个静态变量
 - 重定位：把每个符号定义与一个内存位置关联起来
 - 编译器和汇编器生成从地址0开始的代码和数据节
 - 链接器重定位这些节，修改所有对这些符号的引用，使它们指向这个内存位置
 - 链接器使用汇编器产生的重定位条目的详细指令
- 目标文件是字节块的集合，有些包含代码，有些包含数据，其他的包含引导链接器和加载器的数据结构
 - 链接器将这些块连接起来，确定被连接块的运行时位置，并且修改代码和数据块中的各种位置

7.3 目标文件

- 目标文件有三种类型：
 - 可重定位目标文件（.o文件）
 - 包含二进制代码和数据，其形式可以在编译时与其他可重定位目标文件合并起来，创建一个可执行目标文件
 - 可执行目标文件（a.out文件）
 - 包含二进制代码和数据，其形式可以被直接复制到内存并执行
 - 共享目标文件（.so文件）
 - 一种特殊类型的可重定位目标文件，可以在加载或运行时被动态地加载进内存并连接
- 编译器和汇编器生成可重定位目标文件（包含共享目标文件），链接器生成可执行目标文件
- 一个目标模块是一个字节序列，一个目标文件是一个以文件形式存放在磁盘中的目标模块
- 目标文件的格式：
 - Windows：可移植可执行格式，PE格式
 - MacOS-X：Mach-O格式
 - X86-64 Linux和Unix系统：可执行可链接格式，ELF格式
 - Executable and Linkable Format

单独编译，局部代码



7.4 可重定位目标文件

- ELF头：
 - 以一个16字节的序列开始，描述生成该文件的系统的字的大小和字节顺序
 - ELF头大小、目标文件类型、节头部表的文件偏移、节头部表中条目的大小和数量
 - 不同节的位置和大小由节头部表描述，目标文件中的每个节都有一个固定大小的条目

在ELF头和节头部表之间的都是节，包括：

- Segment header table段头表
 - 页大小、虚拟地址存储段、段大小
- .text section: 已编译程序的机器代码
- .rodata section: 只读数据，如printf语句中的格式串和开关语句的跳转表 *switch*
- .data section: 已初始化的全局和静态C变量
 - 局部C变量在运行时被保存在栈中，既不出现在.data节中，也不出现在.bss节中
- .bss section: 未初始化的全局和静态C变量，以及所有被初始化为0的全局或静态变量
 - 未初始化变量不需要占据任何实际的磁盘空间
 - 运行时在内存中分配这些变量，初始值为0

ELF header
Segment header table (required for executables)
.text section
.rodata section
.data section
.bss section
.symtab section
.rel.txt section
.rel.data section
.debug section
.line section
.strtab section
Section header table

7.4 可重定位目标文件

- **.symtab section:** 符号表，存放在程序中定义和引用的函数和全局变量的信息
 - 每个可重定位目标文件在 .symtab 中都有一张符号表，但不包含局部变量的条目
- **.rel.text section:** .text 节中位置的列表，当链接器把这个目标文件和其他文件组合时，需要修改这个位置
 - 调用外部函数或引用全局变量的指令需要修改
 - 调用本地函数的指令不需要修改
- **.rel.data section:** 被模块引用或定义的所有全局变量的重定位信息
 - 已初始化的全局变量的初始值是一个全局变量地址或外部定义函数的地址，需要被修改
- **.debug section:** 调试符号表，条目是程序中定义的局部变量和类型定义，程序中定义和引用的全局变量，以及原始的C源文件
 - 以选项 -g 调用编译器驱动程序时才会得到该表(gcc -g)
- **.line section:** 原始C源程序中的行号和 .text 节中机器指令之间的映射
 - 以 -g 选项调用编译器驱动程序时才会得到这张表
- **.strtab section:** 字符串表，包括 .symtab 和 .debug 节中的符号表，以及节头部中的节名字
 - 以 -g 选项调用编译器驱动程序时才会得到这张表
- **Section header table:** 每节的偏移地址和大小

ELF header
Segment header table (required for executables)
.text section
.rodata section
.data section
.bss section
.symtab section
.rel.txt section
.rel.data section
.debug section
.line section
.strtab section
Section header table

7.5 符号和符号表

每个可重定位目标模块 m 都有一个符号表，包含 m 定义和引用符号的信息，在链接器的上下文中有三种符号：

- Global symbols全局符号：
 - 由模块 m 定义并能被其他模块引用
 - 非静态C函数（被当前文件以外的其他文件调用，具有全局可见性）和全局变量
- External symbols外部符号：
 - 被其他模块定义并被模块 m 引用的全局符号
 - 在其他模块中定义的非静态C函数和全局变量
- Local symbols局部符号
 - 只被模块 m 定义和引用的局部符号
 - 带**static**属性的C函数和全局变量
 - 在模块 m 中任何位置都可见，但不能被其他模块引用

Static属性本地过程变量

- 本地链接器符号和本地程序变量的不同：
 - 本地程序变量在运行时在栈中管理
 - 带有static属性的本地过程变量不在栈中管理
 - 编译器在.data或.bss中为每个定义分配空间
 - 在符号表中创建一个有唯一名字的本地链接器符号
- 示例：
 - 函数f()和g()中各定义了一个静态局部变量x
 - 编译器向汇编器输出两个不同名字的局部链接器符号
 - X.1表示函数f中的定义
 - X.2表示函数g中的定义
- 用途：利用static属性隐藏变量和函数名
 - 带有static属性声明的全局变量或者函数都是模块私有的
 - 不带static属性声明的全局变量和函数都是公共的，可以被其他模块访问
 - 尽可能用static属性保护变量和函数是好的编程习惯

```
Int f()  
{  
    static int  
    x=0;  
    return x;  
}  
Int g()  
{  
    static int  
    x=1;  
    return x;  
}
```

符号表

- 符号表由汇编器构造，使用编译器输出到汇编语言.s文件中的符号

– .symtab节中包含ELF符号表，包含一个条目的数组，每个条目格式：

- Name: 字符串表中的字节偏移，指向符号的以null结尾的字符串名字
- Type: 数据或函数
- Binding: 本地或全局
- Section: 到节头部表的索引
- Value: 符号的地址，对于可重定位模块来说是距定义目标的节的起始位置的偏移，对可执行目标文件来说是一个绝对运行时地址
- size: 是目标的大小，以字节为单位

```
/* $begin elfsymbol */
typedef struct {
    int    name;        /* String table offset */
    char   type:4,      /* Function or data (4
bits) */
                    binding:4; /* Local or global (4
bits) */
    char   reserved;    /* Unused */
    short  section;     /* Section header index
*/
    long   value;       /* Section offset or
absolute address */
    long   size;        /* Object size in bytes
*/
} Elf64_Symbol;
/* $end elfsymbol */

/* $begin elfrelo */
typedef struct {
    long offset;        /* Offset of the reference
to relocate */
    long type:32,      /* Relocation type */
        symbol:32; /* Symbol table index */
    long addend;       /* Constant part of
relocation expression */
} Elf64_Rela;
/* $end elfrelo */
```

伪节(pseudosection)

- 可重定位目标文件中三个特殊的伪节：
 - ABS: 不该被重定位的符号
 - UNDEF: 未定义的符号，在本目标模块中引用，但却在其他地方定义的符号
 - COMMON: 表示还未被分配位置的未初始化的数据目标
 - COMMON与.bss的区别：
 - COMMON: 未初始化的全局变量
 - .bss: 未初始化的静态变量，以及初始化为0的全局或静态变量

示例：Readelf命令查看目标文件内容

- readelf命令用来显示一个或者多个elf格式的目标文件的信息，可以通过它的选项来控制显示哪些信息。
- 主要选项：
 - a --all Equivalent to: -h -l -S -s -r -d -V -A -I
 - h --file-header Display the ELF file header
 - l --program-headers Display the program headers
 - segments An alias for --program-headers
 - S --section-headers Display the sections' header
 - sections An alias for --section-headers
 - s --syms Display the symbol table
 - symbols An alias for --syms

readelf -a prog (示例)

ELF Header:

```
Magic:  7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
Class:                               ELF64
Data:                               2's complement, little endian
Version:                             1 (current)
OS/ABI:                             UNIX - System V
ABI Version:                          0
Type:                                EXEC (Executable file)
Machine:                             Advanced Micro Devices X86-64
Version:                             0x1
Entry point address:                 0x400464
Start of program headers:            64 (bytes into file)
Start of section headers:            6688 (bytes into file)
Flags:                               0x0
Size of this header:                 64 (bytes)
Size of program headers:             56 (bytes)
Number of program headers:           9
Size of section headers:             64 (bytes)
Number of section headers:           30
Section header string table index: 27
```

Program Headers:

Type	Offset	VirtAddr	PhysAddr
	FileSiz	MemSiz	Flags Align
PHDR	0x0000000000000040	0x0000000000400040	0x0000000000400040
	0x00000000000001f8	0x00000000000001f8	R E 8
INTERP	0x0000000000000238	0x0000000000400238	0x0000000000400238
	0x000000000000001c	0x000000000000001c	R 1
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]			
LOAD	0x0000000000000000	0x0000000000400000	0x0000000000400000
	0x00000000000008ec	0x00000000000008ec	R E 200000
LOAD	0x0000000000000e10	0x0000000000600e10	0x0000000000600e10
	0x000000000000022c	0x0000000000000230	RW 200000
DYNAMIC	0x0000000000000e28	0x0000000000600e28	0x0000000000600e28
	0x00000000000001d0	0x00000000000001d0	RW 8
NOTE	0x0000000000000254	0x0000000000400254	0x0000000000400254
	0x0000000000000044	0x0000000000000044	R 4
GNU_EH_FRAME	0x00000000000007a4	0x00000000004007a4	0x00000000004007a4
	0x000000000000003c	0x000000000000003c	R 4
GNU_STACK	0x0000000000000000	0x0000000000000000	0x0000000000000000
	0x0000000000000000	0x0000000000000000	RW 10
GNU_RELRO	0x0000000000000e10	0x0000000000600e10	0x0000000000600e10
	0x00000000000001f0	0x00000000000001f0	R 1

Section Headers:

[Nr]	Name	Type	Address	Offset
	Size	EntSize	Flags Link Info	Align
[0]		NULL	0000000000000000	00000000
	0000000000000000	0000000000000000	0 0 0	0
[1]	.interp	PROGBITS	0000000000400238	00000238
	000000000000001c	0000000000000000	A 0 0	1
[2]	.note.ABI-tag	NOTE	0000000000400254	00000254
	0000000000000020	0000000000000000	A 0 0	4
[3]	.note.gnu.build-id	NOTE	0000000000400274	00000274
	0000000000000024	0000000000000000	A 0 0	4
[4]	.gnu.hash	GNU_HASH	0000000000400298	00000298
	000000000000001c	0000000000000000	A 5 0	8
[5]	.dynsym	DYNSYM	00000000004002b8	000002b8
	0000000000000060	0000000000000018	A 6 1	8
[6]	.dynstr	STRTAB	0000000000400318	00000318
	000000000000003f	0000000000000000	A 0 0	1
[7]	.gnu.version	VERSYM	0000000000400358	00000358
	0000000000000008	0000000000000002	A 5 0	2
[8]	.gnu.version_r	VERNEED	0000000000400360	00000360
	0000000000000020	0000000000000000	A 6 1	8
[9]	.rela.dyn	RELA	0000000000400380	00000380
	0000000000000018	0000000000000018	A 5 0	8
[10]	.rela.plt	RELA	0000000000400398	00000398
	0000000000000048	0000000000000018	AI 5 12	8
[11]	.init	PROGBITS	00000000004003e0	000003e0
	000000000000001a	0000000000000000	AX 0 0	4
[12]	.plt	PROGBITS	0000000000400400	00000400
	0000000000000040	0000000000000010	AX 0 0	16
[13]	.text	PROGBITS	0000000000400440	00000440
	0000000000000334	0000000000000000	AX 0 0	16

Symbol table '.dynsym' contains 4 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	
1:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	printf@GLIBC_2.2.5 (2)
2:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	_libc_start_main@GLIBC_2.2.5 (2)
3:	0000000000000000	0	NOTYPE	WEAK	DEFAULT	UND	__gmon_start__

Symbol table '.symtab' contains 68 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	
1:	0000000000000238	0	SECTION	LOCAL	DEFAULT	1	
2:	0000000000000254	0	SECTION	LOCAL	DEFAULT	2	
3:	0000000000000274	0	SECTION	LOCAL	DEFAULT	3	
4:	0000000000000298	0	SECTION	LOCAL	DEFAULT	4	
5:	00000000000002b8	0	SECTION	LOCAL	DEFAULT	5	
6:	0000000000000318	0	SECTION	LOCAL	DEFAULT	6	
7:	0000000000000358	0	SECTION	LOCAL	DEFAULT	7	
8:	0000000000000360	0	SECTION	LOCAL	DEFAULT	8	
9:	0000000000000380	0	SECTION	LOCAL	DEFAULT	9	
10:	0000000000000398	0	SECTION	LOCAL	DEFAULT	10	
11:	00000000000003e0	0	SECTION	LOCAL	DEFAULT	11	
12:	0000000000000400	0	SECTION	LOCAL	DEFAULT	12	

练习题 7.1

- 针对m.o和swap.o模块，每个在swap.o中定义或引用的符号指出其是否在模块swap.o中的.symtab节中有一个符号条目，如果是，指出定义该符号的模块、符号类型以及它在模块中被分配到的节。

符号	.symtab条目?	符号类型	在哪个模块中定义	节	说明
buf	是	外部	main.o	.data	已初始化的全局变量
bufp0	是	全局	swap.o	.data	已初始化的全局变量
bufp1	是	全局	swap.o	COMMON	未初始化的全局变量
swap	是	全局	swap.o	.text	已编译程序的机器代码
temp	否	——	——	——	——

```
Void swap();
int buf[2]={1,2};
int main()
{
    swap();
    return 0;
}
//m.c
```

```
Extern int buf[];
Int *bufp0=&buf[0];
Int *bufp1;
Void swap()
{
    int temp;
    bufp1=&buf[1];
    temp=*bufp0;
    *bufp0=*bufp1;
    *bufp1=temp;
}
//swap.c
```

作业1：

HOME
WORK

- P497-7.6
- 关于图7-5的m.o模块和给定的swap.c函数版本指出符号相关信息
- 目标：强化练习题7.1

作业2:



- P499-7.12
- 考虑目标文件m.o中对函数swap的调用
- 目标：扩展作业7.6

7.6 符号解析

- 链接器解析符号引用的方法：将每个引用与它输入的可重定位目标文件的符号表中的一个确定的符号定义关联起来
- 当引用定义在相同模块中的局部符号引用时：
 - 编译器只允许每个模块中每个局部符号有一个定义
 - 静态局部变量有本地链接器符号，编译器确保它们有唯一的名字
- 不在当前模块中定义的符号：
 - 假设该符号在其他某个模块中定义，生成一个链接器符号表条目，把它交给链接器处理
 - 若链接器在它的任何输入模块中都找不到这个被引用符号的定义，输出错误信息并终止
 - 示例：链接器解析foo引用错误
 - `gcc -Wall -O3 -o linkerror linkerror.c`（注：-Wall显示警告信息）

```
void foo(void);
int main(){
    foo();
    return 0;
}
//linkerror.c
```

```
[nsrc@localhost ~]$ gcc -Wall -O3 -o linkerror linkerror.c
/tmp/ccqrnEWN.o: In function `main':
linkerror.c(.text.startup+0x5): undefined reference to `foo'
collect2: error: ld returned 1 exit status
```

7.6.1 链接器如何解析多重定义的全局符号

- 问题：链接器的输入是一组可重定位目标模块，每个模块定义一组符号，有局部的，有全局的，当多个模块定义同名的全局符号时，如何处理？
- 编译时，编译器向汇编器输入每个全局符号，或强或弱
 - 函数和已初始化的全局变量是强符号
 - 未初始化的全局变量是弱符号
- 根据强弱符号定义，链接器处理多重定义符号的规则：
 - 规则1：不允许有多个同名的强符号
 - 规则2：如果有一个强符号和多个弱符号同名，则选择强符号
 - 规则3：如果有多个弱符号同名，则从这些弱符号中任意选择一个
- 当两个模块使用相同的函数名main时，链接器生成错误信息
 - gcc foo1.c bar1.c
 - multiple definition of `main`, 强符号main被定义多次（规则1）

```
int main()
{
    return 0;
}
//foo1.c
```

```
int main()
{
    return 0;
}
//bar1.c
```

```
[nsrc@localhost ~]$ gcc foo1.c bar1.c
/tmp/ccyv8Ruq.o: In function `main':
bar1.c:(.text+0x0): multiple definition of `main'
/tmp/ccFQM9CH.o:foo1.c:(.text+0x0): first defined here
collect2: error: ld returned 1 exit status
```

链接器规则示例

- 全局变量强符号x多次定义：规则1

```
int x=15213;
int main()
{
    return 0;
}
//foo2.c
```

```
int x=15213;
void f()
{
}
//bar2.c
```

```
[nsrc@localhost ~]$ gcc foo2.c bar2.c
/tmp/ccq97kH6.o:(.data+0x0): multiple definition of `x'
/tmp/cceXwTPF.o:(.data+0x0): first defined here
```

- 若一个模块里x未初始化，则链接器选择另一个模块中定义的强符号：规则2

— 执行强符号f()函数赋值15212

```
#include <stdio.h>
void f(void);
int x=15213;
int main() ←①
{
    f();
    printf("x=%d\n",
x);
    return 0;
}
//foo3.c
```

```
#include <stdio.h>
int x;
void f()
{
    printf("x=%d\n",x);

x=15212; ←②
}
//bar3.c
```

```
[nsrc@localhost ~]$ gcc -o foobar3 foo3.c bar3.c
[nsrc@localhost ~]$ foobar3
x=15213
x=15212
```

链接器规则示例

- X有两个弱定义：规则3

- 两个全局变量x均未赋值，f为强符号，x取f函数值

```
#include <stdio.h>
void f(void);
int x;
int main()
{
    x=15213;
    f();
    printf("x=%d\n", x);
    return 0;
}
//foo4.c
```

```
int x;
void f()
{
    x=15212;
}
//bar4.c
```

```
[nsrc@localhost ~]$ gcc -o foobar4 foo4.c bar4.c
[nsrc@localhost ~]$ foobar4
x=15212
```

链接器规则示例

- X有两个不同数据类型定义：规则2、规则3
 - 重复的符号定义有不同的数据类型
 - gcc -o foobar5 foo5.c bar5.c
 - Double为8字节，int为4字节，f函数赋值后覆盖内存x，y的位置

```
#include <stdio.h>
void f(void);
int y=15212;
int x=15213;
int main()
{
    f();
    printf("x=0x%x[%d] y=0x%x[%d]
\n",x,&x,y,&y); return 0;
}
//foo5.c
```

```
double x;
void f()
{
    x=-0.0;
}
//bar5.c
```

```
[nsrc@localhost ~]$ gcc -Wall -O3 -o foobar5 foo5.c bar5.c
foo5.c: In function ? . ain? .
foo5.c:8:2: warning: format ? . d? . expects argument of type ? . nt? . but argument 3 has type ? . nt *? . [-Wformat=]
    printf("x=0x%x[addr:%d] y=0x%x[addr:%d] \n",x,&x,y,&y);
    ^
foo5.c:8:2: warning: format ? . d? . expects argument of type ? . nt? . but argument 5 has type ? . nt *? . [-Wformat=]
/usr/bin/ld: Warning: alignment 4 of symbol `x' in /tmp/ccHqTrwB.o is smaller than 8 in /tmp/ccnWgKgM.o
[nsrc@localhost ~]$ foobar5
x=0x0[addr:6295604] y=0x80000000[addr:6295608]
```

练习题7.2

- REF(x.i)→DEF(x.k)表示链接器将把模块i中对符号x的任意引用与模块k中x的定义关联起来。分析下例链接器如何解析每个模块中对多重定义符号的引用

A.

```
/*module
1*/
Int main()
{
}
```

```
/*module
2*/
Int main;
Int p2()
{
}
```

按规则2，选择模块1中的强符号

(a) REF(main.1) → DEF(main.1)

(b) REF(main.2) → DEF(main.1)

B.

```
/*module
1*/
void main()
{
}
```

```
/*module
2*/
Int main=1;
Int p2()
{
}
```

错误，违反规则1，每个模块都定义了一个强符号

(a) REF(main.1) → DEF(,)

(b) REF(main.2) → DEF(,)

C.

```
/*module
1*/
Int x;
Void main()
{
}
```

```
/*module
2*/
Double
x=1.0;
Int p2()
{
}
```

按规则2，选择模块2中的强符号

(a) REF(x.1) → DEF(x.2)

(b) REF(x.2) → DEF(x.2)

7.6.2 与静态库链接

- 静态库(static library): 编译系统将所有相关的目标模块打包成为一个单独的文件
 - 用做链接器的输入
 - 当链接器构造一个输出的可执行文件时, 只复制静态库里被应用程序引用的目标模块
- 静态库主要解决标准C函数模块链接到目标可执行文件的复本代价和标准函数更新问题
- 静态库工作原理:
 - 相关函数被编译为独立的目标模块, 然后封装成一个单独的静态库文件
 - 应用程序可以通过在命令行上指定单独的文件名字来使用这些在库中定义的函数
 - Linux系统中, 静态库以一种文档(archive)文件格式存放在磁盘中, 存档文件是一组连接起来的可重定位目标集合, 有一个头部用来描述每个成员大小和位置, 文件后缀为.a

静态库示例

```
gcc -c addvec.c multvec.c
ar rcs libvector.a addvec.o multvec.o
```

libvector.a

```
int addcnt = 0;

void addvec(int *x, int *y,
            int *z, int n) {
    int i;

    addcnt++;
    for (i = 0; i < n; i++)
        z[i] = x[i] + y[i];
}
```

addvec.c

```
int multcnt = 0;

void multvec(int *x, int *y,
             int *z, int n)
{
    int i;

    multcnt++;
    for (i = 0; i < n; i++)
        z[i] = x[i] * y[i];
}
```

multvec.c

```
gcc -c main2.c
gcc -static -o prog2c main2.o ./libvector.a
或
gcc -c main2.c
gcc -static -o prog2c main2.o -L. -lvector
```

```
#ifndef VECTOR_H
#define VECTOR_H
void addvec(int *x, int *y, int *z,
            int n);
void multvec(int *x, int *y, int *z,
             int n);
#endif
#include <stdio.h>
#include "vector.h"
int x[2] = {1, 2};
int y[2] = {3, 4};
int z[2];
int main(){
    multvec(x, y, z, 2);
    printf("z = [%d %d]\n", z[0], z[1]);
    addvec(x, y, z, 2);
    printf("z = [%d %d]\n", z[0], z[1]);
    return 0;
}
```

main2.c



```
[nsrc@localhost ~]$ gcc -c main2.c
[nsrc@localhost ~]$ gcc -static -o prog2c main2.o ./libvector.a
[nsrc@localhost ~]$ prog2c
z = [3 8]
z = [4 6]
[nsrc@localhost ~]$ gcc -c main2.c
[nsrc@localhost ~]$ gcc -static -o prog2c main2.o -L. -lvector
[nsrc@localhost ~]$ prog2c
z = [3 8]
z = [4 6]
```

与静态库链接器行为图

源文件

addvec.c multvec.c

Translators
(gcc -c)

可重定位目标文件

addvec.o multvec.o

源文件

main2.c vector.h

Translators
(cpp, cc1, as)

Archiver
(ar)

libvector.a

libc.a 静态库

printf.o和其他
printf.o调用的模块

可重定位目标文件

main2.o

addvec.o

Linker (ld)

prog2c 完全链接的可执行目标文件

7.6.3 链接器如何使用静态库来解析引用

- 符号解析阶段，链接器从左到右按照它们在编译器驱动程序命令行上出现的顺序来扫描可重定位目标文件和存档文件，扫描中，链接器维护三个集合：
 - 可重定位目标文件集合 E ：集合中的文件会被合并起来形成可执行文件
 - 未解析符号集合 U ：引用了但是尚未定义的符号
 - 符号集合 D ：在前面输入文件中已定义的符号集合
 - 初始时， E 、 U 、 D 均为空
- 解析过程：
 - 命令行上的每个输入文件 f ，链接器判断 f 是一个目标文件还是一个存档文件
 - 如果 f 是一个目标文件，链接器把 f 添加到 E ，修改 U 和 D 来反映 f 中的符号定义和引用，并继续下一个输入文件
 - 如果 f 是一个存档文件，链接器尝试匹配 U 中未解析的符号和由存档文件成员定义的符号
 - 如果某个存档文件成员 m 定义了一个符号来解析 U 中的一个引用，则将 m 加到 E 中，并且链接器修改 U 和 D 来反映 m 中的符号定义和引用
 - 对存档文件中所有的成员目标文件都进行这个过程，直到 U 和 D 不再发生变化
 - 任何不包含在 E 中的成员目标文件都被简单地抛弃，链接器继续处理下一个输入文件
 - 如果链接器完成对命令行上输入文件的扫描后， U 是非空的，则链接器输出一个错误并终止；否则会合并和重定位 E 中的目标文件，构建输出的可执行文件

算法可能导致的链接错误

- 算法要求命令行上的库和目标文件顺序性
 - 符号库的出现在引用这个符号的目标文件之前时，引用不能被解析，链接失败

```
[nsrc@localhost ~]$ gcc -static ./libvector.a main2.c  
/tmp/cc8hCVei.o: In function `main':  
main2.c:(.text+0x19): undefined reference to `multvec'  
main2.c:(.text+0x4f): undefined reference to `addvec'  
collect2: error: ld returned 1 exit status
```

- 在处理libvector.a时，U是空的，没有libvector.a中的成员目标文件会添加到E中，对addvec的引用不会被解析，链接器产生一条错误信息并终止
- 库的一般准则是将它们放在命令行的结尾
 - 如果各个库的成员相互独立，这些库可以以任意顺序放置在命令行结尾
 - 如果库不相互独立，需要对它们排序，使每个被存档文件的成员外部引用的符号s在命令行中至少有一个s的定义是在对s的引用之后
 - 如：foo.c→libx.a, foo.c→libz.a, libx.a→liby.a, libz.a→liby.a，则命令句中libx.a和libz.a必须处在liby.a之前，即 gcc foo.c libx.a libz.a liby.a
 - 如果需要满足依赖需求，可以在命令行上重复库
 - 如：foo.c→libx.a, foo.c→liby.a, liby.a→libx.a, libx.a必须在命令行上重复出现，即 gcc foo.c libx.a liby.a libx.a
 - 或者，将libx.a和liby.a合并成一个单独的存档文件

练习题7.3

- a和b表示当前目录中的目标模块或者静态库， $a \rightarrow b$ 表示a依赖于b，也就是说b定义了一个被a引用的符号。对于下面每种场景，请给出最小的命令行，即一个含有最少数量的目标文件和库参数的命令，使得静态链接器能解析所有的符号引用
- A. $p.o \rightarrow libx.a$
 - gcc p.o libx.a
- B. $p.o \rightarrow libx.a \rightarrow liby.a$
 - gcc p.o libx.a liby.a
- C. $p.o \rightarrow libx.a \rightarrow liby.a$ 且 $liby.a \rightarrow libx.a \rightarrow p.o$
 - gcc p.o libx.a liby.a libx.a

7.7 重定位

- 链接器完成符号解析后，把代码中的每个符号引用和相应的一个符号定义(即它的一个输入目标模块中的一个符号表条目)关联起来。链接器此时知道它的输入目标模块中的代码节和数据节的确切大小，开始重定位步骤，将合并输入模块，并为每个符号分配运行时地址
- 重定位步骤：
 - 重定位节和符号定义：链接器将所有相同类型的节合并为同一类型的新的聚合节
 - 来自所有输入模块的.data节被全部合并成一个节，成为输出的可执行目标文件的.data节
 - 链接器将运行时内存地址赋给新的聚合节，赋给输入模块定义的每个节，以及赋给输入模块定义的每个符号
 - 程序中的每条指令和全局变量都有唯一的运行时内存地址
 - 重定位节中的符号引用：链接器修改代码节和数据节中对每个符号的引用，使得它们指向正确的运行时地址
 - 链接器依赖于可重定位目标模块中称为重定位条目的数据结构

7.7.1 重定位条目

- 汇编器生成一个重定位条目，告诉链接器在将目标文件合并成可执行文件时如何修改这个引用
 - 代码重定位条目放在`.rel.text`中，已初始化数据的重定位条目放在`.rel.data`中
 - ELF重定位条目格式：

- Offset是需要被修改的引用的节偏移
- Symbol标识被修改引用应该指向的符号
- Type告知链接器如何修改新的引用
- Addend是一个有符号常数，一些类型的重定位要使用它对被修改引用的值做偏移调整

```
/* $begin elfrelo */
typedef struct {
    long offset;      /* Offset of the reference to relocate */
    long type:32,     /* Relocation type */
        symbol:32; /* Symbol table index */
    long addend;      /* Constant part of relocation expression */
} Elf64_Rela;
/* $end elfrelo */
```

- ELF定义了32种不同的重定位类型，其中两种最基本的重定位类型：
 - R_X86_64_PC32：重定位一个使用32位PC相对地址的引用
 - 一个PC相对地址就是距程序计数器的当前运行时值的偏移量，CPU执行一条使用PC相对寻址指令时将指令中编码的32位值加上PC当前运行时址，得到有效地址
 - R_X86_64_32：重定位一个使用32位绝对地址的引用，CPU使用在指令中编码的32位值作为有效地址绝对寻址

重定位算法

1. Foreach section s{
2. foreach relocation entry r{
3. refptr=s+r.offset;
4. /*重定位32位PC相对地址引用*/
5. if(r.type==R_X86_64_PC32){
6. refaddr=ADDR(s)+r.offset;
7. *refptr=(unsigned)(ADDR(r.symbol)+r.addend-refaddr);}
8. /*重定位32位绝对地址引用*/
9. if(r.type==R_X86_64_32){
10. *refptr=(unsigned)(ADDR(r.symbol)+r.addend);
11. /*重定位32位绝对地址引用*/

- 假设算法运行时，链接器为每个节(ADDR(s))和每个符号(ADDR(r.symbol))都选择了运行时地址
 - 第3行计算的是需要被重定位的4字节引用的数组s中的地址
 - PC相对寻址使用5~7行重定位
 - 绝对寻址使用8~10行重定位

7.7.2 重定位符号引用

```
1. 0000000000000000 <main>:
2. 0:      48 83 ec 08          sub    $0x8,%rsp
3. 4:      be 02 00 00 00      mov     $0x2,%esi
4. 9:      bf 00 00 00 00      mov     $0x0,%edi    %edi=$array
5.                a: R_X86_64_32 array          relocation entry
6. e:      e8 00 00 00 00      callq  13 <main+0x13>    sum()
7.                f: R_X86_64_PC32 sum-0x4      relocation entry
8. 13:     48 83 c4 08          add     $0x8,%rsp
9. 17:     c3                  retq
```

18: sum ~

```
int sum(int *a, int n);

int array[2] = {1, 2};

int main()
{
    int val = sum(array, 2);
    return val;
}

main.c
```

- main函数引用两个全局符号：array引用使用32位绝对地址进行重定位；sum引用使用32位PC相对地址进行重定位

1. 重定位相对引用

- Sum函数call指令开始于节偏移0xe处理，包括一字节操作码0xe8，后面跟着对目标sum的32位PC相对引用占位符

- 重定位条目r由4个字段组成：

- r.offset=0xf r.symbol=sum r.type=R_X86_64_PC32 r.addend=-4

- 假设链接器已确定ADDR(S)=ADDR(.text)=0x4004d0和ADDR(r.symbol)=ADDR(sum)=0x4004e8

- 链接器计算出引用的运行时地址：

- refaddr=ADDR(s)+r.offset=0x4004d0+0xf=0x4004df

- 更新该引用，使它运行时指向sum程序

- *refptr=(unsigned)(ADDR(r.symbol)+r.addend-refaddr)=0x4004e8+(-4)-0x4004df=0x5

- 在得到的可执行目标文件中，call指令有如下的重定位形式：

- 指令存放在4004de: e8 05 00 00 00 callq 4004e8 <sum> sum

- CPU执行call指令时PC值为0x4004e3，执行指令计数后得到PC+0x5→PC=0x4004e8

```
int sum(int *a, int n)
{
    int i, s = 0;

    for (i = 0; i < n; i++)
        s += a[i];
    return s;
}

sum.c
```

7.7.2 重定位符号引用

• 2.重定位绝对引用

```
1. 0000000000000000 <main>:
2. 0:      48 83 ec 08          sub   $0x8,%rsp
3. 4:      be 02 00 00 00      mov   $0x2,%esi
4. 9:      bf 00 00 00 00      mov   $0x0,%edi    %edi=$array
5.                a: R_X86_64_32  array          relocation entry
6. e:      e8 00 00 00 00      callq 13 <main+0x13>    sum()
7.                f: R_X86_64_PC32 sum-0x4      relocation entry
8. 13:     48 83 c4 08          add   $0x8,%rsp
9. 17:     c3                  retq
```

```
int sum(int *a, int n);

int array[2] = {1, 2};

int main()
{
    int val = sum(array, 2);
    return val;
}

main.c
```

- Mov指令将array的地址复制到寄存器%edi中，mov指令始于节偏移量0x9的位置，包括1字节操作码0xbf，后面跟着对array的32位绝对引用的占位符

- r.offset=0xa r.symbol=array r.type=R_X86_64_32 r.addend=0
- 链接器要修改从偏移量0xa开始的绝对引用，运行时指向array的第一个字节，假设链接器已确定ADDR(r.symbol)=ADDR(array)=0x601018
- 链接器修改引用：
- *refptr=(unsigned) (ADDR(r.symbol)+r.addend)
- =(unsigned) (0x601018+0)
- =(unsigned) (0x601018)

- 得到的可执行目标，该引用有下面的重定位形式：

- 4004d9: bf 18 10 60 00 mov \$0x601018,%edi

```
int sum(int *a, int n)
{
    int i, s = 0;

    for (i = 0; i < n; i++)
        s += a[i];

    return s;
}

sum.c
```

最终可执行目标文件中已重定位的.text节和.data节

ADDR(S)

重定位的.text节

1.	00000000004004d0 <main>:	
2.	4004d0:	48 83 ec 08 sub \$0x8,%rsp
3.	4004d4:	be 02 00 00 00 mov \$0x2,%esi
4.	4004d9:	bf 18 10 60 00 mov \$0x0,%edi <i>%edi=\$array</i>
5.	4004de:	e8 00 00 00 00 callq 4004e8 <sum>
6.	4004e3:	48 83 c4 08 add \$0x8,%rsp
7.	4004e7:	c3 ADDR(array) retq
8.	00000000004004e8 <sum>:	ADDR(r.symbol)
9.	4004e8:	b8 00 00 00 00 mov \$0x0,%eax
10.	4004ed:	ba 00 00 00 00 mov \$0x0,%edx
11.	4004f2:	eb 09 jmp 4004fd <sum+0x15>
12.	4004f4:	48 63 ca movslq %edx, %rcx
13.	4004f7:	03 04 8f add (%rdi,%rcx,4),%eax
14.	4004fa:	83 c2 01 add \$0x1,%edx
15.	4004fd:	39 f2 cmp %esi,%edx
16.	4004ff:	7c f3 jl 4004f4 <sum+0xc>
17.	400501:	f3 c3 repz retq

重定位的.data节

1.	0000000000601018 <array>:	
2.	601018:	01 00 00 00 02 00 00 00

int array[2] = {1, 2};

练习题7.4

- A.第5行中对sum的重定位引用的十六进制地址是多少？
 - 地址是：0x4004df(**0x4004de+1**)
- B.第5行中对sum的重定位引用的十六进制值是多少？
 - 值是0x5

```
1. 00000000004004d0 <main>:
2. 4004d0: 48 83 ec 08          sub    $0x8,%rsp
3. 4004d4: be 02 00 00 00      mov    $0x2,%esi
4. 4004d9: bf 18 10 60 00      mov    $0x0,%edi    %edi=$array
5. 4004de: e8 05 00 00 00      callq 4004e8 <sum>
6. 4004e3: 48 83 c4 08          add    $0x8,%rsp
7. 4004e7: c3                  retq
```

练习题7.5

- 考虑目标文件m.o中对swap函数的调用
- 9: e8 00 00 00 00 callq e <main+0xe> swap
- 它的重定位条目如下:
 - r.offset=0xa r.symbol=swap
 - r.type=R_X86_64_PC32 r.addend=-4
- 假设链接器将m.o中的.text重定位到地址0x4004d0, 将swap重定位到地址0x4004e8, callq指令中对swap重定位引用的值是什么?
 - ADDR(s)=ADDR(.text)=0x4004d0
 - ADDR(r.symbol)=ADDR(swap)=0x4004e8
 - 计算引用的运行时地址:
 - refaddr=ADDR(s)+r.offset= 0x04004d0+0xa=0x04004da
 - 修改此引用:
 - *refptr=(unsigned) (ADDR(r.symbol)+r.addend-refaddr
 - =(unsigned) (0x4004e8+(-4)- 0x04004da)=(unsigned) (0xa)
 - 得到可执行目标文件中, 对swap的PC相对引用的值为0xa:
 - 4004d9: e8 0a 00 00 00 callq 4004e8 <swap>

```
Void swap();
int buf[2]={1,2};
int main()
{
    swap();
    return 0;
}
//m.c
```

```
Extern int buf[];
Int *bufp0=&buf[0];
Int *bufp1;
Void swap()
{
    int temp;
    bufp1=&buf[1];
    temp=*bufp0;
    *bufp0=*bufp1;
    *bufp1=temp;
}
//swap.c
```

7.8 可执行目标文件

- 典型ELF可执行文件中的各类信息
 - ELF头：描述文件的总体格式，包括程序入口点，即当程序运行时要执行的第一条指令的地址
 - .init节：定义了一个叫做_init的小函数，程序的初始化代码会调用它
 - .text, .rodata, .data节与可重定位目标文件中的节类似，这些节已被重定位到它们最终的运行时内存地址
 - 可执行文件是完全链接的(已被重定位)，不再需要.rel节
 - 程序头部表：描述可执行文件连续的片与连续的内存段的映射关系

可执行目标文件

ELF头
程序头部表
.init section
.text section
.rodata section
.data section
.bss section
.symtab
.debug
.line
.strtab
节头部表 (重定位用)

可执行文件程序头部表

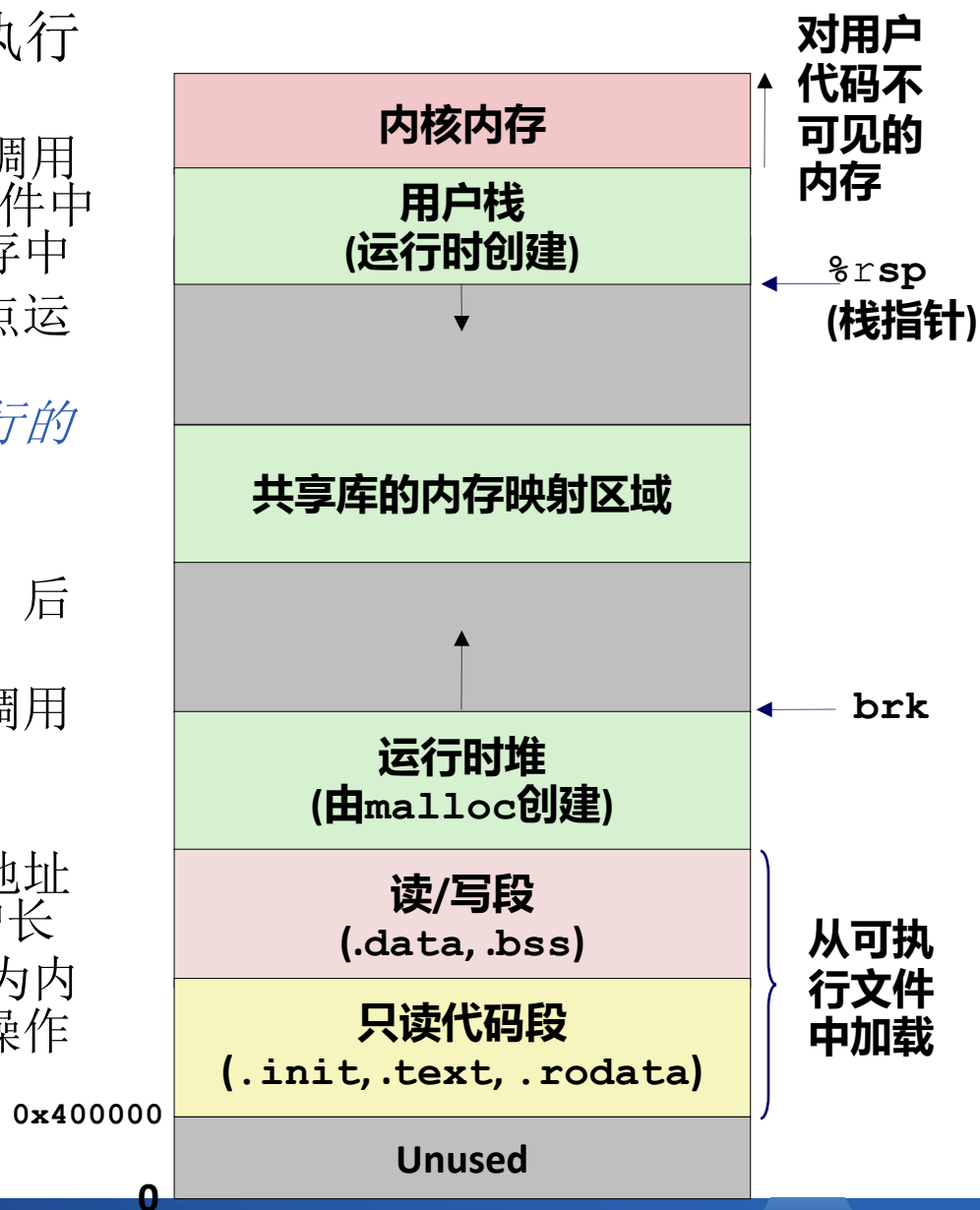
- 从程序头部表中可以看到可执行目标文件的内容初始化为两个内存段：
 - 第一个代码段有读/执行访问权限(flags r-x)，目标文件中的偏移为0(off)，开始于内存地址0x400000(vaddr)，内存中的段大小是0x69c字节(memsz)，被初始化为可执行目标文件的头0x69c个字节(filesz)，包括ELF头、程序头部表以及.init、.text和.rodata节
 - 第二个数据段有读/写访问权限(flags rw-)，开始于内存地址0x600df8处，总的内存大小为0x230字节，并用从目标文件中偏移0xdf8处开始的.data节中的0x228个字节初始化
 - 任何段s，链接器必须选择一个起始地址vaddr，使 $vaddr \bmod align = off \bmod align$
 - $vaddr \bmod align = 0x600df8 \bmod 0x200000 = 0xdf8$
 - $off \bmod align = 0xdf8 \bmod 0x200000 = 0xdf8$

Program Header:

```
PHDR off 0x0000000000000040 vaddr 0x0000000000400040 paddr 0x0000000000400040 align 2**3
      filesz 0x00000000000001f8 memsz 0x00000000000001f8 flags r-x
INTERP off 0x0000000000000238 vaddr 0x0000000000400238 paddr 0x0000000000400238 align 2**0
      filesz 0x000000000000001c memsz 0x000000000000001c flags r--
LOAD off 0x0000000000000000 vaddr 0x0000000000400000 paddr 0x0000000000400000 align 2**21
      filesz 0x000000000000069c memsz 0x000000000000069c flags r-x
LOAD off 0x0000000000000df8 vaddr 0x0000000000600df8 paddr 0x0000000000600df8 align 2**21
      filesz 0x0000000000000228 memsz 0x0000000000000230 flags rw-
DYNAMIC off 0x0000000000000e10 vaddr 0x0000000000600e10 paddr 0x0000000000600e10 align 2**3
      filesz 0x00000000000001d0 memsz 0x00000000000001d0 flags rw-
NOTE off 0x0000000000000254 vaddr 0x0000000000400254 paddr 0x0000000000400254 align 2**2
      filesz 0x0000000000000044 memsz 0x0000000000000044 flags r--
EH_FRAME off 0x00000000000005b4 vaddr 0x00000000004005b4 paddr 0x00000000004005b4 align 2**2
      filesz 0x0000000000000034 memsz 0x0000000000000034 flags r--
STACK off 0x0000000000000000 vaddr 0x0000000000000000 paddr 0x0000000000000000 align 2**3
      filesz 0x0000000000000000 memsz 0x0000000000000000 flags rw-
RELRO off 0x0000000000000df8 vaddr 0x0000000000600df8 paddr 0x0000000000600df8 align 2**0
      filesz 0x0000000000000208 memsz 0x0000000000000208 flags r--
```

7.9 加载可执行目标文件

- 在linux shell命令行中输入可执行目标文件prog时：
 - Linux程序通过调用execve函数调用加载器loader，将可执行目标文件中的代码和数据从磁盘复制到内存中
 - 跳转到程序第一条指令或入口点运行该程序
 - 加载：将程序复制到内存并运行的过程*
- 程序运行时内存映像
 - 代码段总是从0x400000处开始，后面是数据段
 - 运行时堆在数据段之后，通过调用malloc库向上增长
 - 堆后面的区域为共享模块保留
 - 用户栈总是从最大的合法用户地址($2^{48}-1$)开始，向较小内存地址增长
 - 栈上的区域，从地址 2^{48} 开始是为内核中的代码和数据保留的，是操作系统驻留在内存的部分



加载器运行执行过程

- 加载器运行时：
 - 创建内存映像
 - 在程序头部表的引导下，将可执行文件的片复制到代码段和数据段
 - 加载器跳转到程序的入口点，即 `_start` 函数的地址，此函数在系统目标文件 `ctrl.o` 中定义
 - `_start` 函数调用系统启动函数 `__libc_start_main`，该函数定义在 `libc.so` 中
 - 初始化执行环境，调用用户层的 `main` 函数，处理 `main` 函数的返回值，并在需要时把控制返回给内核

SYMBOL TABLE:

```
0000000000400238 l d .interp 0000000000000000 .interp
0000000000400254 l d .note.ABI-tag 0000000000000000 .note.ABI-tag
0000000000400274 l d .note.gnu.build-id 0000000000000000 .note.gnu.build-id
0000000000000000 F *UND* 0000000000000000 __libc_start_main@@GLIBC_2.2.5
0000000000601008 g .data 0000000000000000 __data_start
0000000000000000 w *UND* 0000000000000000 __gmon_start__
```

Disassembly of section .init:

```
0000000000400390 <_init>:
400390: 48 83 ec 08      sub    $0x8,%rsp
400394: e8 53 00 00 00   callq 4003ec <call_gmon_start>
400399: 48 83 c4 08      add    $0x8,%rsp
40039d: c3              retq
```

Disassembly of section .plt:

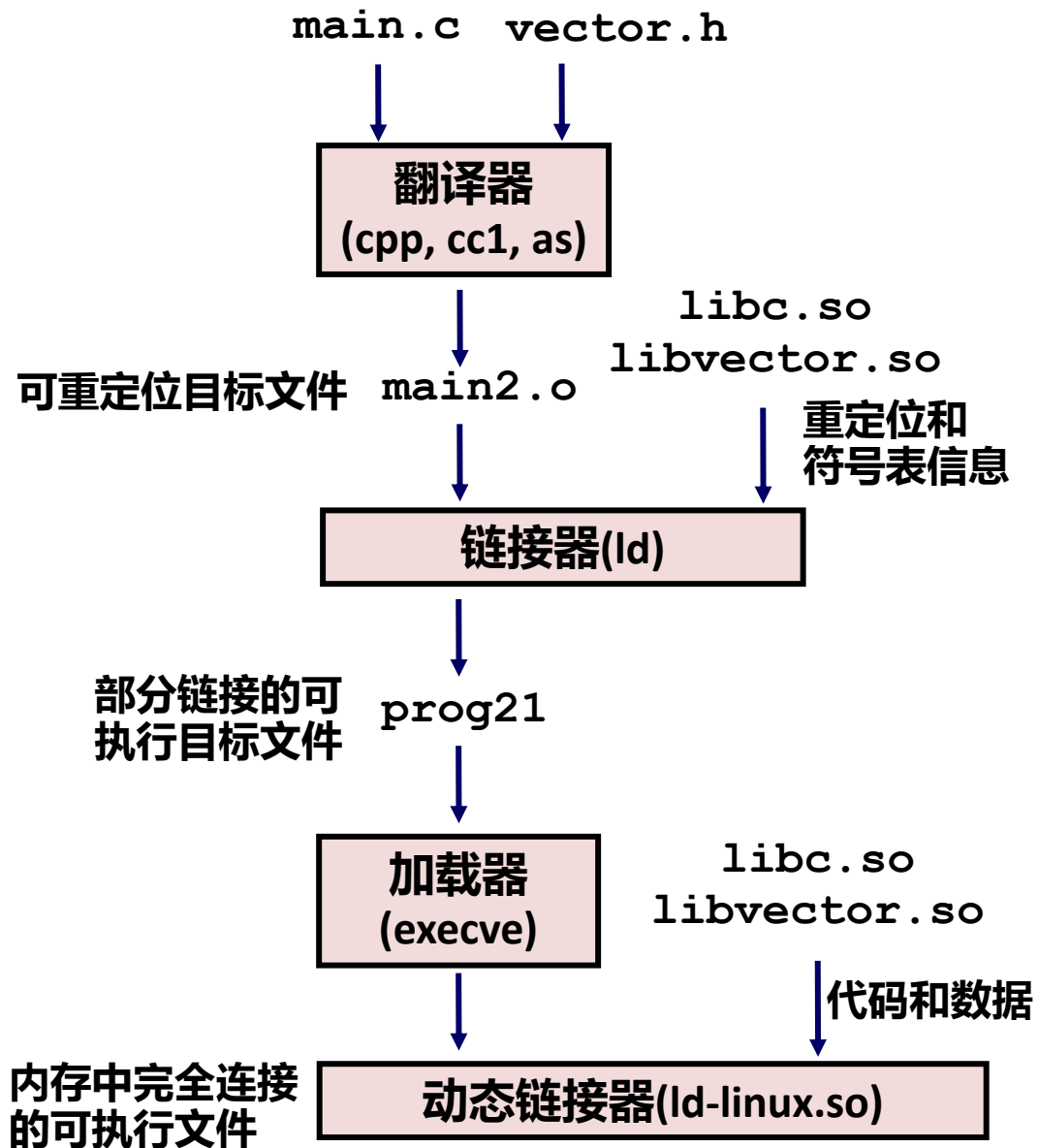
```
00000000004003a0 <__libc_start_main@plt-0x10>:
4003a0: ff 35 4a 0c 20 00 pushq 0x200c4a(%rip) # 600ff0 <_GLOBAL_OFFSET_TABLE_+0x8>
4003a6: ff 25 4c 0c 20 00 jmpq *0x200c4c(%rip) # 600ff8 <_GLOBAL_OFFSET_TABLE_+0x10>
4003ac: 0f 1f 40 00      nopl 0x0(%rax)

00000000004003b0 <__libc_start_main@plt>:
4003b0: ff 25 4a 0c 20 00 jmpq *0x200c4a(%rip) # 601000 <_GLOBAL_OFFSET_TABLE_+0x18>
4003b6: 68 00 00 00 00   pushq $0x0
4003bb: e9 e0 ff ff ff   jmpq 4003a0 <_init+0x10>
```

7.10 动态链接共享库

- 静态库缺点：
 - 定期维护和更新：需要应用程序员以显式的方式将程序与更新的库重新链接
 - 标准I/O函数代码重复复制到每个运行进程的文本段中，浪费内存空间
- 共享库(shared library)：是一个目标模块，在运行或加载时可以加载到任意的内存地址，并和一个在内存中的程序链接起来。
 - 加载共享库的过程称为动态链接(dynamic linking)
 - 动态链接由动态链接器(dynamic linker)的程序执行
 - 共享库也称为共享目标(shared object)
 - 在Linux系统中通常用.so后缀表示
 - 在Windows系统中称为DLL(动态链接库)
- 共享库的两种共享方式：
 - 任何给定的系统中，一个库只有一个.so文件，所有引用该库的可执行目标文件共享这个.so文件中的代码和数据，不用复制和嵌入引用它们的可执行文件中
 - 内存中，一个共享库的.text节的一个副本可以被不同的正在运行的进程共享

动态链接过程示例



构造向量例程共享库

- 为向量例程创建共享库:

- 创建共享库libvector.so:

- gcc -shared -fpic -o libvector.so addvec.c multvec.c

- -fpic选项指示编译器生成与位置无关的代码

- -shared选项指示链接器创建一个共享的目标文件

- 将共享库链接到示例程序:

- gcc -o prog21 main2.c ./libvector.so

- 创建了可执行目标文件prog21, 在运行时可以和libvector.so链接

```
[nsrc@localhost ~]$ gcc -shared -fpic -o libvector.so addvec.c multvec.c
[nsrc@localhost ~]$ ls
5.12          bar2.c        foo2.c        load.c        perf.data.old  sum.i         twiddle
5.12.c        bar3.c        foo3.c        main2.c       Pictures       sum.o        twiddle.c
6.18          bar4.c        foo4.c        main2.o       poly          sum.s        vec
6.18.c        bar5.c        foo5.c        main.c        poly.c        swap         vec3
6.6.1         cpu_simd      foobar3       main.i        prefixsum     swap.c       vec3.c
6.6.1.c       cpu_SIMD.c    foobar4       main.o        prefixsum.c   Templates   vec.c
6.6.1-vectorjoin  cup_simd      foobar5       main.s        prog          test5.3     vec.h
6.6.1-vectorjoin.c Desktop      gmon.out      merge         Public        test5.3.c   vec.s
6.6.2         Documents     inlinefcall   merge.c       Rdtsc.h      test6.8     vector.h
6.6.2-0       doumul        inlinefcall.c monetdb        rearrange     test6.8.c   Videos
6.6.2-0.c     doumul.c     libvector.a   multvec.c     rearrange.c   timer       timer.c
6.6.2.c       Downloads    libvector.so  multvec.o     slow.c        timertest   timetest.c
addvec.c       fcall        linkererror.c nsrc2017      slower.c      TPC-H_100
addvec.o       fcall.c      load          perf.data    sum.c
bar1.c         foo1.c
[nsrc@localhost ~]$ gcc -o prog21 main2.c ./libvector.so
[nsrc@localhost ~]$ prog21
z = [3 8]
z = [4 6]
```

文件执行过程解析

- 创建可执行文件时，静态执行一些链接，然后在程序加载时动态完成链接过程
 - 没有任何libvector.so的代码和数据节被复制到可执行文件prog21中
 - 链接器复制了一些重定位和符号表信息，运行时可以解析对libvector.so中代码和数据的引用
- 加载器加载和运行可执行文件prog21
 - 加载部分链接的可执行文件prog21
 - Prog21包含的.interp节中包含动态链接器的路径名(动态链接器ld-linux.so)
 - 加载器加载和运行动态链接器
 - 动态链接器通过执行下面的重定位完成链接任务：
 - 重定位libc.so的文本和数据到某个内存段
 - 重定位libvector.so的文本和数据到另一个内存段
 - 重定位prog21中所有对由libc.so和libvector.so定义的符号的引用
 - 动态链接器将控制传递给应用程序，共享库的位置固定下来，在程序执行过程中不会改变

7.11 从应用程序中加载和链接共享库

- 运行时加载和链接共享库
 - 应用场景：应用程序在运行时要求动态链接器加载和链接某个共享库，无需在编译时将那些库链接到应中
 - 应用示例：
 - 分发软件：微软Windows应用利用共享库分发软件更新，当用户下载共享库新版本后，在运行应用程序时自动链接和加载新的共享库
 - 构建高性能web服务器：现代高性能web服务器使用基于动态链接的方法生成动态内容
 - 将每个生成动态内容的函数打包在共享库中，web请求到达时服务器动态地加载和链接适当的函数，直接调用它

Linux为动态链接器提供的接口

- Linux系统提供动态链接器接口，允许应用程序在运行时加载和链接共享库

- dlopen函数:

```
#include <dlfcn.h>
Void *dlopen (const char *filename, int
flag);
```

- 返回: 若成功则为指向句柄的指针，若出错则为NULL
- 功能: 加载和链接共享库filename
- 选项:
 - RTLD_GLOBAL: 用已用RTLD_GLOBAL选项打开了的库解析filename中的外部符号
 - -rdynamic: 使用该选项编译的可执行文件，对符号解析而言，它的全局符号也是可用的
 - Flag: RTLD_NOW|RTLD_LAZY
 - » RTLD_NOW: 告诉链接器立即解析对外部符号的引用
 - » RTLD_LAZY: 指标链接器推迟符号解析直到执行来自库中的代码

- dlsym函数:

```
#include <dlfcn.h>
Void *dlsym (void *handle, char
*symbol);
```

- 返回: 若成功则为指向符号的指针，若出错则为NULL
- 功能: 输入一个指向前面已打开了的共享库的句柄和一个symbol名字，如果该符号存在，就返回符号地址，否则返回NULL

- dlclose函数:

```
#include <dlfcn.h>
Void *dlclose (void
*handle);
```

- 返回: 若成功则为0，若出错则为-1
- 功能: 如果没有其他共享库使用这个共享库，则卸载该共享库

- dlerror函数:

```
#include <dlfcn.h>
Void *dlerror
(void);
```

- 返回: 如果前面对dlopen、dlsym或dlclose的调用失败，则为错误消息，如果前面的调用成功，则为NULL
- 功能: 返回描述调用dlopen、dlsym或dlclose函数时发生的最近的错误

动态链接共享库示例

- 编译程序(示例):
 - gcc -rdynamic -ldl -o prog2r dll.c
- 主要函数使用解析:
 - handle=dlopen("./libvector.so",RTLD_LAZY);
 - 获得libvector.so共享库句柄,推迟符号解析直到执行来自库中的代码
 - addvec=dlsym(handle,"addvec");
 - 返回符号addvec的地址
 - dlclose(handle)
 - 卸载该共享库

```
#include <stdio.h>
#include <stdlib.h>
#include <dlfcn.h>
int x[2]={1,2};
int y[2]={3,4};
int z[2];
int main()
{
    void *handle;
    void (*addvec)(int *, int *, int *, int);
    char *error;
    /*dynamically load the shared library containing
addvec()*/
    handle=dlopen("./libvector.so",RTLD_LAZY);
    if(!handle){
        fprintf(stderr,"%s\n",dlerror());
        exit(1);
    }
    /*get a pointer to the addvec() function we just
loaded*/
    addvec=dlsym(handle,"addvec");
    if((error=dlerror())!=NULL){
        fprintf(stderr,"%s\n",error);
        exit(1);
    }
    /*now we call addvec() just like any other
function*/
    addvec(x,y,z,2);
    printf("z=[%d %d]\n",z[0],z[1]);
    /*unload the shared library*/
    if(dlclose(handle)<0){
        fprintf(stderr,"%s\n",dlerror());
        exit(1);
    }
    return 0;
}
```

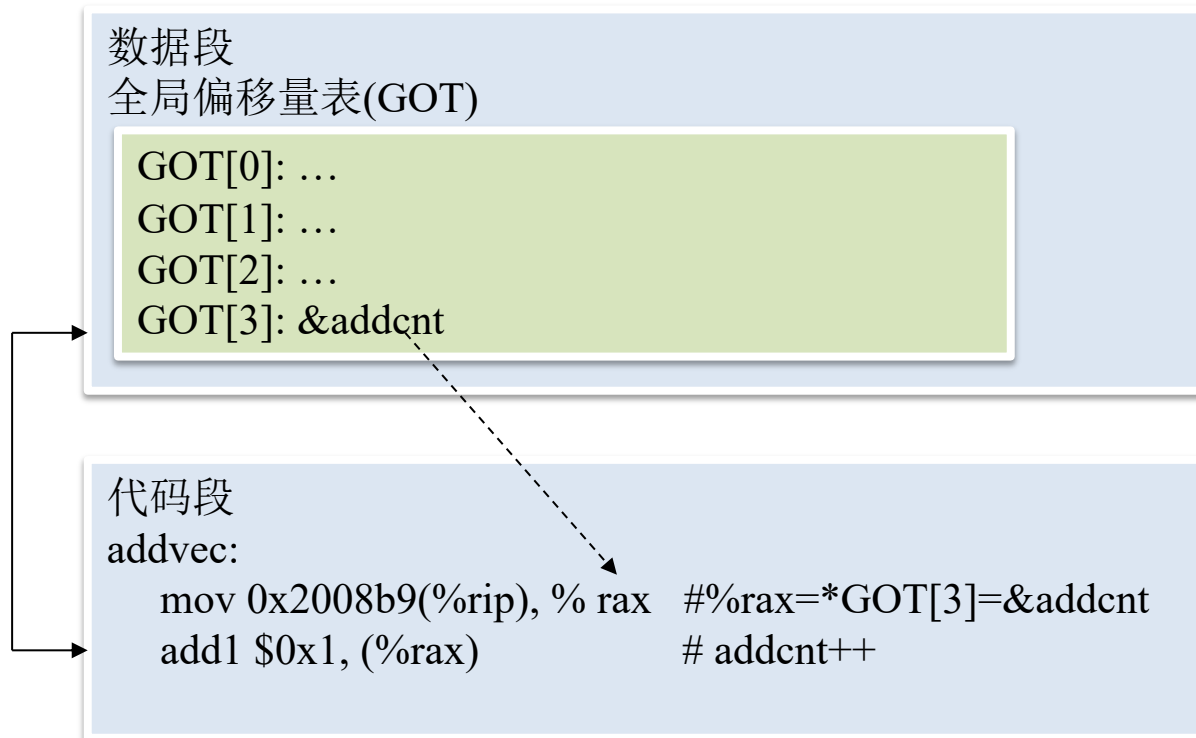
7.12 位置无关代码

- 动机：为共享库预分配专用地址空间片，管理复杂，成本高
- 目标：编译共享模块代码段，使得可以把它们加载到内存的任何位置而无需链接器修改，使多个进程可以共享一个共享代码段的单一副本
- 位置无关代码(Position-Independent Code, PIC):
 - 可以加载而无需重定位的代码称为位置无关代码
 - GCC使用-fpic选项指示GNU编译系统生成PIC代码，共享库编译必须总是使用该选项
- 1. PIC数据引用
 - 技术基础：数据段与代码段的距离保持不变，代码段中的任何指令和数据段中任何变量之间的距离都是一个运行时常量，与代码段和数据段的绝对内存位置无关
 - 方法：在数据段开始的地方创建一个全局偏移量表(Global Offset Table, GOT)
 - 每个被这个目标模块引用的全局数据目标(过程或全局变量)都有一个8字节条目，编译器为GOT中每个条目生成一个重定位记录
 - 加载时，动态链接器重定位GOT中的每个条目，使得它包含目标的正确的绝对地址
 - 每个引用全局目标的目标模块都有自己的GOT

GOT示例

- 示例libvector.so共享模拟GOT:
 - addvec例程通过GOT[3]间接地加载全局变量addcnt地址，然后把addcnt在内存中加1
 - 增加一个重定位，让链接器在构造这个共享模块时解析它

运行时GOT[3]和add1
指令之间的固定距离
是0x2008b9



7.12 位置无关代码

• 2. PIC函数调用

- 延迟绑定(lazy binding): 将过程地址的绑定推迟到第一次调用该过程时
 - 典型应用只使用共享库函数中很少的一部分通过将函数地址的解析推迟到它实际被调用的地方, 避免动态链接器在加载时进行成百上千个并不需要的重定位
- 实现方法: 两个数据结构之间的交互
 - GOT和过程链接表(Procedure Linkage Table, PLT): 如果一个目标模块调用定义在共享库中的任何函数, 那么它就有自己的GOT和PLT, GOT是数据段的一部分, PLT是代码段的一部分
 - 过程链接表(PLT): 是一个每个条目16字节代码构成的数组
 - PLT[0]: 特殊条目, 跳转到动态链接器中, 每个被可执行程序调用的库函数都有它自己的PLT条目
 - PLT[1]: 调用系统函数__libc_start_main, 初始化执行环境, 调用main函数并处理其返回值
 - PLT[2]: 从此开始的条目调用用户代码调用的函数
 - 全局偏移量表(GOT): 一个每条目是8字节地址的数组
 - 和PLT联合作用时, GOT[0]和GOT[1]包含动态链接器在解析函数地址时会使用的信息
 - GOT[2]: 动态链接器在ld-linux.so模块中的入口点
 - 其余每个条目都有一个相匹配的PLT条目, 初始时, 每个GOT条目都指向对应PLT条目的第二条指令

GOT和PLT协同工作

- 第一次调用addvec时，延迟解析运行时地址：
 - 第1步：不直接调用addvec，程序调用进入PLT[2]，即addvec的PLT条目
 - 第2步：第一条PLT指令通过GOT[4]进行间接跳转，每个GOT条目初始时指向它对应的PLT条目的第二条指令，间接跳转把控制传送回PLT[2]中的下一条指令
 - 第3步：把addvec的ID(0x1)压入栈中之后，PLT[2]跳转到PLT[0]
 - 第4步：PLT[0]通过GOT[1]间接地把动态链接器的一个参数压入栈中，然后通过GOT[2]间接跳转进动态链接器中，动态链接器使用两个栈条目来确定addvec的运行位置，用这个地址重写GOT[4]，再把控制传递给addvec

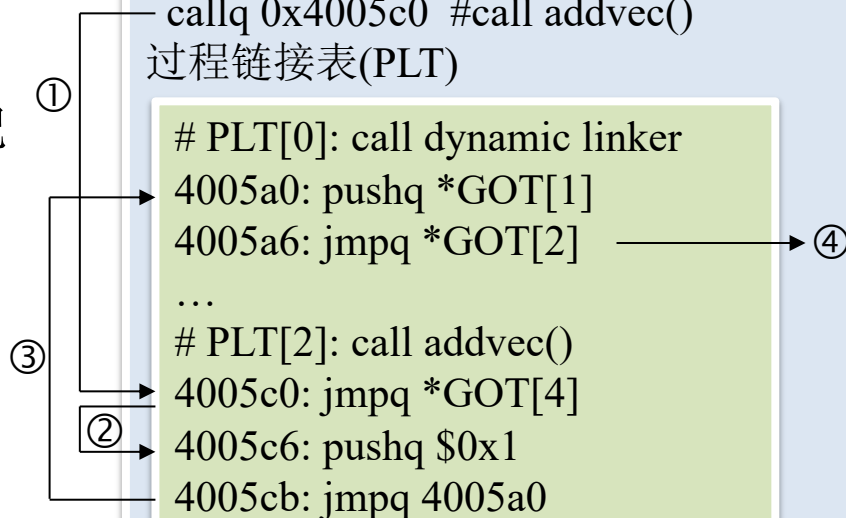
数据段
全局偏移量表(GOT)

```
GOT[0]: addr of .dynamic
GOT[1]: addr of reloc entries
GOT[2]: addr of linker
GOT[3]: 0x4005b6 #sys startup
GOT[4]: 0x4005c6 # addvec()
GOT[5]: 0x4005d6 # printf()
```

代码段

callq 0x4005c0 #call addvec()
过程链接表(PLT)

```
# PLT[0]: call dynamic linker
4005a0: pushq *GOT[1]
4005a6: jmpq *GOT[2]
...
# PLT[2]: call addvec()
4005c0: jmpq *GOT[4]
4005c6: pushq $0x1
4005cb: jmpq 4005a0
```



GOT和PLT协同工作

- 后续调用addvec时的控制流：
 - 第1步：控制传递到PLT[2]
 - 第2步：通过GOT[4]的间接跳转将控制直接转移到addvec

数据段

全局偏移量表(GOT)

```
GOT[0]: addr of .dynamic
GOT[1]: addr of reloc entries
GOT[2]: addr of linker
GOT[3]: 0x4005b6 #sys startup
GOT[4]: &addvec()
GOT[5]: 0x4005d6 # printf()
```

代码段

callq 0x4005c0 #call addvec()

过程链接表(PLT)

```
# PLT[0]: call dynamic linker
4005a0: pushq *GOT[1]
4005a6: jmpq *GOT[2]
...
# PLT[2]: call addvec()
4005c0: jmpq *GOT[4]
4005c6: pushq $0x1
4005cb: jmpq 4005a0
```

①

②

7.13 库打桩机制

- 库打桩(library interpositioning): Linux链接器允许用户截获对共享库函数的调用, 取而代之执行自己的代码
 - 作用: 可以追踪对某个特殊库函数的调用次数, 验证和追踪它的输入和输出值, 将它替换成一个完全不同的实现
- 基本思想:
 - 给定一个需要打桩的目标函数, 创建一个包装函数, 它的原型与目标函数完全一样
 - 通过某种特殊的打桩机制, 可以欺骗系统调用包装函数替代目标函数
 - 包装函数通常会执行它自己的逻辑, 然后调用目标函数, 再将目标函数的返回值传递给调用者
- 打桩发生的时机: 编译、链接或当程序被加载和执行的运行时

7.13.1 编译时打桩

- mymalloc.c中的包装函数调用目标函数，打印追踪记录；本地malloc.h头文件指示预处理器用对应包装函数的调用替换对目标函数的调用
- 参数-I.进行打桩，告诉C预处理器先在当前目录查找malloc.h

- gcc -DCOMPILETIME -c mymalloc.c
- gcc -I. -o intc int.c mymalloc.o
- ./intc

```
[nsrc@localhost ~]$ ./intc
malloc(32)=0x2088010
free (0x2088010)
```

```
#include <stdio.h>
#include <malloc.h>
```

```
int main()
{
    int *p = malloc(32);
    free(p);
    return(0);
}
/* int.c*/
```

```
#define malloc(size) mymalloc(size)
#define free(ptr) myfree(ptr)
void *mymalloc(size_t size);
void myfree(void *ptr);
/* malloc.h*/
```

```
#ifndef COMPILETIME
#include <stdio.h>
#include <malloc.h>
/*malloc wrapper function*/
void *mymalloc(size_t size)
{
    void *ptr=malloc(size);
    printf("malloc(%d)=%p\n",(int)size, ptr);
    return ptr;
}
/*free wrapper function*/
void myfree(void *ptr)
{
    free(ptr);
    printf("free (%p)\n",ptr);
}
#endif /* mymalloc.c*/
```

7.13.2 链接时打桩

- Linux静态链接器支持用—wrap f标志进行链接时打桩，该标志告诉链接器将对符号f的引用解析为__wrap_f，把对符号__real_f的引用解析为f

- gcc -DLINKTIME -c
mymalloc_wrap.c
- gcc -c int.c
- gcc -Wl,--wrap,malloc -Wl,--
wrap,free -o int1 int.o
mymalloc_wrap.o
- ./int1

```
[nsrc@localhost ~]$ ./int1
malloc(32)=0xbc0010
free (0xbc0010)
```

```
#ifndef LINKTIME
#include <stdio.h>
void *__real_malloc(size_t size);
void __real_free(void *ptr);
/*malloc wrapper function*/
void *__wrap_malloc(size_t size)
{
    void *ptr=__real_malloc(size);// call libc malloc
    printf("malloc(%d)=%p\n",(int)size, ptr);
    return ptr;
}
/*free wrapper function*/
void __wrap_free(void *ptr)
{
    __real_free(ptr);//call libc free
    printf("free (%p)\n",ptr);
}
#endif
/* mymalloc.c*/
```

7.13.3 运行时打桩

- 三种打桩方式的条件：
 - 编译时打桩：需要能够访问程序的源代码
 - 链接时打桩：需要能够访问程序的可重定位对象文件
 - 运行时打桩：只需要能够访问可执行目标文件
 - 原理：LD_PRELOAD环境变量指示共享库搜索路径，动态链接器LD-LINUX.SO先搜索LD_PRELOAD库

运行时打桩示例

- 在malloc和free包装函数中，对dlsym的调用返回指向目标libc函数的指针，包装函数调用目标函数，打印追踪记录，再返回
- 构建包含包装函数共享库的方法：
 - gcc -DRUNTIME -shared -fpic -o mymalloc_ld.so mymalloc_ld.c -ldl
- 编译主程序：
 - gcc -o intr int.c
- 运行程序：
 - LD_PRELOAD="./mymalloc_ld.so" ./intr
- LD_PRELOAD对任何可执行程序库函数调用打桩
 - LD_PRELOAD="./mymalloc_ld.so" /usr/bin/uptime

```
#ifdef RUNTIME
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <dlfcn.h>
/*malloc wrapper function*/
void *malloc(size_t size)
{
    void *(*mallocp)(size_t size);
    char *error;
    mallocp=dlsym(RTLD_NEXT,"malloc");//get address of libc
    malloc
    if((error=dLError())!=NULL){
        fputs(error,stderr);
        exit(1);
    }
    char *ptr=mallocp(size);//call libc malloc
    printf("malloc(%d)=%p\n",(int)size, ptr);
    return ptr;
}
/*free wrapper function*/
void free(void *ptr)
{
    void (*freep)(void *)=NULL;
    char *error;
    if(!ptr) return;
    freep=dlsym(RTLD_NEXT,"free");//get address of libc free
    if((error=dLError())!=NULL){
        fputs(error,stderr);
        exit(1);
    }
    freep(ptr);//call libc free
    printf("free (%p)\n",ptr);
}
#endif
/* mymalloc.c*/
```

```
[nsr@localhost ~]$ gcc -o intr int.c
[nsr@localhost ~]$ LD_PRELOAD="./mymalloc_ld.so" ./intr
malloc(32)=0x13df010
free (0x13df010)
```

```
[nsr@localhost ~]$ LD_PRELOAD="./mymalloc_ld.so" /usr/bin/uptime
malloc(37)=0x21e4040
malloc(16)=0x21e4070
malloc(568)=0x21e4090
malloc(4096)=0x21e42d0
free (0x21e4090)
free (0x21e42d0)
```

7.14 处理目标文件的工具

- AR: 创建静态库, 插入、删除、列出和提取成员
- STRINGS: 列出一个目标文件中所有可打印的字符串
 - strings intr
- STRIP: 从目标文件中删除符号表信息
- NM: 列出一个目标文件的符号表中定义的符号
- SIZE: 列出目标文件中节 名字和大小
- READELF: 显示一个目标文件的完整结构, 包括ELF头中编码的所有信息
- OBJDUMP: 显示一个目标文件中所有的信息, 最大的作用是反汇编.text节中的二进制指令
- LDD: 列出一个可执行文件在运行时所需要的共享库

```
[nsrc@localhost ~]$ ldd intr
linux-vdso.so.1 => (0x00007fff4fbef000)
libc.so.6 => /lib64/libc.so.6 (0x00007f5471fe4000)
/lib64/ld-linux-x86-64.so.2 (0x00007f54723c5000)
```

```
[nsrc@localhost ~]$ strings intr
/lib64/ld-linux-x86-64.so.2
libc.so.6
malloc
__libc_start_main
free
__gmon_start__
GLIBC_2.2.5
```

```
[nsrc@localhost ~]$ nm intr
000000000060103c B __bss_start
000000000060103c b completed.6344
0000000000601038 D __data_start
0000000000601038 W data_start
00000000004004c0 t deregister_tm_clones
0000000000400530 t __do_global_ctors_aux
0000000000600e18 t __do_global_ctors_aux_fini_array_entry
0000000000400638 R __dso_handle
0000000000600e28 d _DYNAMIC
000000000060103c D _edata
0000000000601040 B _end
0000000000400624 T _fini
0000000000400550 t frame_dummy
0000000000600e10 t __frame_dummy_init_array_entry
0000000000400768 r _FRAME_END
U free@@GLIBC_2.2.5
0000000000601000 d _GLOBAL_OFFSET_TABLE_
w __gmon_start__
0000000000400418 T _init
0000000000600e18 t __init_array_end
0000000000600e10 t __init_array_start
0000000000400630 R _IO_stdin_used
w _ITM_deregisterTMCloneTable
w _ITM_registerTMCloneTable
0000000000600e20 d _JCR_END
0000000000600e20 d _JCR_LIST__
w _Jv_RegisterClasses
0000000000400620 T __libc_csu_fini
00000000004005b0 T __libc_csu_init
U __libc_start_main@@GLIBC_2.2.5
000000000040057d T main
U malloc@@GLIBC_2.2.5
00000000004004f0 t register_tm_clones
0000000000400490 T _start
0000000000601040 D _TMC_END__
```

```
[nsrc@localhost ~]$ size intr
text    data    bss     dec      hex filename
1305    556      4      1865     749 intr
```

```
[nsrc@localhost ~]$ readelf -S intr
There are 30 section headers, starting at offset 0x19f8:

Section Headers:
 [Nr] Name              Type              Address            Offset
     Size            EntSize          Flags              Link Info Align
 [ 0]                   NULL              0000000000000000   0 0 0
 [ 1] .interp            PROGBITS          0000000000040238   00000238
                000000000000001c   A 0 0 1
 [ 2] .note.ABI-tag      NOTE              0000000000040254   00000254
                0000000000000020   A 0 0 4
 [ 3] .note.gnu.build-id NOTE              0000000000040274   00000274
                0000000000000024   0000000000000000   A 0 0 4
 [ 4] .gnu.hash           GNU_HASH          0000000000040298   00000298
                000000000000001c   0000000000000000   A 5 0 8
```

7.15 小结

- 链接的两种方式：
 - 编译时由静态编译器完成
 - 加载时和运行时由动态链接器完成
- 链接器处理称为目标文件的二进制文件有3种方式：
 - 可重定位的、可执行的和共享的
- 链接器的两个主要任务：
 - 符号解析：将目标的每个全局符号都绑定到一个唯一的定义
 - 重定位：确定每个符号的最终内存地址，并修改对那些目标的引用
- 静态链接器由GCC这样的编译驱动程序调用，将多个可重定位目标文件合并成一个单独的可执行目标文件
- 多个目标文件可以被连接到一个单独的静态库中，链接器用库来解析其他目标模块中的符号引用
- 加载器将可执行文件的内容映射到内存，并运行这个程序；链接器还可能生成部分链接的可执行目标文件，文件中有对定义在共享库中的例程和数据的未解析的引用
- 加载时，加载器将部分链接的可执行文件映射到内存，然后调用动态链接器，通过加载共享库和重定位程序中的引用来完成链接任务
- 被编译为位置无关代码的共享库可以加载到任何地方，也可以在运行时被多个进程共享；为了加载、链接和访问共享库的函数和数据，应用程序也可以在运行时使用动态链接器



THANK YOU
谢谢